

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Duration :60 Mins

Total Marks:50

Assessment Mode: In Class

Annexure A

Scenario Based Task

Code of Conduct:

*I ___K.Madhulika(192524190)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: **Madhulika**

Q. No	Understanding of Concepts (25)	Experimental Design (25)	Use of Tools (20)	Analysis of Results (20)	Documentation (10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 25	/ 20	/ 20	/ 10	/ 100

Annexure A

Scenario Based Task

1. A compiler team is developing a syntax-directed translation module for evaluating expressions and generating intermediate code simultaneously. The grammar contains inherited and synthesized attributes, and the implementation uses an LR parser. During testing, expressions involving nested arithmetic operations produce correct syntax trees but incorrect intermediate code because inherited attributes are evaluated after reductions occur. The developers attempt to fix the issue by introducing temporary variables, but circular attribute dependencies begin to appear in the dependency graph. As a result, some expressions are evaluated repeatedly while others generate incomplete code. The compiler also fails for nested parenthesized expressions and produces different results depending on the order in which parser actions are executed.

Task:

Analyze the attribute dependency graph and determine whether the current syntax-directed definition is S-attributed, L-attributed, or neither. Identify the source of the circular dependency and explain why inherited attributes are difficult to evaluate directly during bottom-up parsing. Propose a redesigned translation scheme that preserves operator precedence, eliminates circular dependencies, and supports efficient intermediate code generation during LR parsing. Justify whether your solution should use marker non-terminals, explicit semantic stacks, or a top-down translation strategy.

2. A compiler constructs syntax trees for arithmetic expressions and later performs common subexpression elimination. For the expression $(a + b) * (a + b) - (a + b)$, the compiler generates three separate $a + b$ nodes instead of reusing a single shared subtree. Consequently, the intermediate code contains redundant computations and prevents optimization. When the optimization phase attempts to merge identical subtrees, the compiler mistakenly combines structurally different expressions that contain type-converted operands, producing incorrect results. The developers discover

that the tree comparison routine checks only operator symbols and ignores operand types and evaluation contexts.

Task:

Analyze the syntax tree construction and optimization process. Explain why structurally identical subexpressions should be shared while expressions with different type information should remain separate. Identify the flaw in the subtree comparison algorithm and redesign the syntax tree representation so that semantic information is preserved during optimization. Propose a debugging strategy that verifies both structural equivalence and type compatibility before eliminating common subexpressions. Generate the optimized three-address code for the given expression and justify the reduction in computational complexity.

3. A compiler for a statically typed language supports integers, floating-point values, characters, arrays, and user-defined record types. The type checker accepts assignments between unrelated record types and silently performs implicit conversions from floating-point values to integers inside complex expressions. During execution, programs produce incorrect numerical results and memory corruption when arrays of incompatible record types are assigned. Investigation shows that the compiler uses name equivalence for variables but structural equivalence for expressions, resulting in inconsistent type checking across different language constructs. In addition, the type checker performs implicit conversions before verifying operand compatibility.

Task:

Analyze the type checking algorithm and identify the inconsistencies between name equivalence and structural equivalence. Explain how the order of type checking and type conversion affects semantic correctness. Redesign the type checker so that compatibility is verified before any implicit conversion occurs and type equivalence is applied consistently throughout the language. Justify whether widening and narrowing conversions should be handled differently for assignments and arithmetic expressions. Illustrate the corrected type checking process using a program that contains arrays, records, and mixed-type arithmetic expressions.

4. A compiler uses bottom-up translation to generate three-address code and subsequently converts it into target machine instructions. For expressions involving nested multiplication and division, the generated code produces unnecessary temporary variables and excessive register movements. Example: $x = (a * b) / (c * d)$ generates six temporary variables and multiple redundant MOV instructions, causing register spilling and increased execution time. The debugging logs reveal that every reduction creates a new temporary variable even when an existing register could be reused. Furthermore, the code generator ignores the lifetime of temporary values and fails to perform simple register allocation.

Task:

Analyze the interaction between bottom-up translation and the back-end code generator. Identify why unnecessary temporaries and register movements are produced during reductions. Redesign the translation and code generation strategy to minimize temporary variables, reuse registers safely, and preserve correctness for nested expressions. Explain how attribute evaluation order influences register allocation decisions. Generate optimized three-address code and corresponding target instructions for the given expression, and compare the number of temporaries, instructions, and register transfers before and after debugging.

5. A predictive parser is integrated with a semantic analyzer that computes FIRST and FOLLOW sets and propagates type attributes during parsing. The grammar contains epsilon productions and declarations of variables with different data types. During debugging, the parser incorrectly selects productions whenever epsilon appears in multiple alternatives, and the semantic analyzer reports undeclared-variable errors even for correctly declared identifiers. Investigation reveals that FOLLOW sets are computed correctly, but inherited type attributes are propagated before the parser confirms the selected production. As a result, incorrect type environments are created and semantic actions execute in the wrong order.

Task:

Analyze the predictive parsing and semantic analysis process. Explain how FIRST and FOLLOW sets influence production selection in the presence of epsilon productions.

Identify the error in inherited attribute propagation and describe how it affects the symbol table and type environment. Redesign the parsing and semantic action execution order so that declarations, type attributes, and identifier usage are processed consistently. Justify whether an L-attributed definition is sufficient for this grammar or whether additional translation mechanisms are required. Demonstrate the corrected parsing and type propagation process for a declaration and assignment sequence involving multiple data types.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

CO3 AT3

Name: K. Madhulika

Reg no: 192524190

Course code: CSA 1419

Course name: Compiler Design

1. Nature of the Attribute Definition:

The given syntax-directed definition contains both inherited and synthesized attributes.

→ Synthesized attributes are calculated from the attributes of child nodes and are naturally suitable for bottom-up parsing.

→ Inherited attributes obtain information from the parent or left-side siblings and normally require information to be available before a production is reduced.

Therefore, the current definition is not S-attribute because S-attribute definitions contain only synthesized attributes.

It can be L-attribute only if every inherited attribute of a symbol depends only on:

1. Attributes of its parent, and

2. Attributes of symbols to its left.

However, the problem states that inherited attributes are evaluated after reductions and that temporary variables introduce circular dependencies.

Therefore, the current definition is best classified as neither S-attributed nor L-attributed, because circular dependencies exist in the attribute dependency graph.

Source of circular Dependency:

The circular dependency occurs when an inherited attribute depends on an attribute directly or indirectly depends back on the same inherited attribute

For example:

A-inherited $\rightarrow$ B-synthesized $\rightarrow$ A-inherited

This cycle creates a dependency graph.

The developers use of temporary variables does not solve the dependency problem because the temporary values themselves may depend on attributes that have not yet been evaluated.

As a result:

- $\rightarrow$ Some expressions are evaluated repeatedly.
- $\rightarrow$ Some intermediate code is incomplete.
- $\rightarrow$ The result may depend on the order of semantic actions.
- $\rightarrow$ Nested expressions and parentheses may fail.

LR parsing works using bottom-up parsing.

Therefore, directly evaluating inherited attributes during LR parsing can cause:

- $\rightarrow$ Dependency problems

- Incorrect evaluation order
- Circular dependencies
- Repeated evaluation
- Difficult handling of nested expressions.

Maker non-terminals are not necessary for the basic solution.

For this problem, it is simpler and safer to:

- use synthesized attributes.
- use explicit semantic values on the semantic stack.
- Generate code when reduction occurs.

Therefore, explicit semantic stacks are preferred.

The original SDD is neither S-attributed nor L-attributed because its dependency graph contains circular dependencies. Inherited attributes are difficult to evaluate directly during LR parsing works bottom-up, while inherited attributes often requires information from the parent or left context.

The best solution is to redesign the translation scheme using synthesized attributes, an explicit semantic stack and three address code generation during LR reductions.

2. Given, expression

$$(a+b) * (a+b) - (a+b)$$

The subexpression $a+b$ occurs 3 times.

This shared subtree can be used by all three operations. This technique is called common subexpression elimination (CSE).

Two expressions may look structurally similar but have different types.

For example:

$$a(\text{int}) + b(\text{int})$$

$$a(\text{float}) + b(\text{float})$$

Although both have the same operator $+$, their meanings and results are different.

Expressions should be shared only when:

- Operators are the same.
- Operands are same.
- Operand types are the same.
- Type conversions are the same.
- Evaluation context is compatible.

The existing comparison routine checks only the operator symbols.

For example, it may compare:

$a+b$ and $(\text{float})a+b$ as identical because both contain the $+$ operator.

This is incorrect.

Correct comparison should check:

operator + operand identity + operand type + type conversion.

Debugging Strategy:

- Compare all operators.
- Compare the left and right operands.
- Check operand datatypes.
- Check type conversions.
- Check evaluation context.
- Share the subtree only when all checks match.
- Test with both identical and type-converted expressions.

Common subexpression elimination should share structurally and semantically identical subtrees, but expressions with different types or conversions must remain separate. The comparison algorithm should check both structure and type information before merging subtrees.

3. Type Checking and Type Equivalence:

Problem:

The compiler uses name equivalence for variables and structural equivalence for expressions.

→ Name equivalence: Two types are equal only if they have the same declared type name.

→ Structural equivalence: Two types are equal if their structure is the same.

using both methods inconsistently can cause incorrect assignments.

Correct Type Checking Algorithm:

- Determine the type of each operand.
- Check whether the types are compatible.
- Apply the language's equivalence rule consistently.
- If compatible, perform an allowed conversion.
- If incompatible, report as type error.
- Never compatible convert an incompatible type just to make the operation valid.

The main error is inconsistent type equivalence and performing conversion before compatibility checking. The corrected compiler should use one consistent equivalence rule, preferably name equivalence for user-defined records and arrays, check compatibility first, and then apply conversions. Widening conversions can generally be implicit, while narrowing conversions should require explicit conversion to prevent loss of data and incorrect results.

4. Bottom-up Translation and Register Allocation:

Problem:

The compiler creates a new temporary variable for every reduction during bottom-up parsing. This causes:

- More temporary variables
- More instructions
- More register usage
- Register spilling
- Increased execution time

Improved Translation Strategy:

- Generate a temporary only when necessary.
- Keep frequently used values in registers.
- Reuse a register after its previous value is no longer needed.
- Track the lifetime of each temporary.
- Generate machine instructions directly from the optimized three-address code when possible.

Optimized Three-Address code:

$$t_1 = a \times b$$

$$t_2 = c \times d$$

$$t_3 = t_1 / t_2$$

$$x = t_3$$

only three temporaries are needed, and t_3 can be eliminated by directly assigning the result:

$$t_1 = a * b$$

$$t_2 = c * d$$

$$x = t_1 / t_2$$

Optimized Target Instructions:

MOV R1, a

~~MOV~~ ^{MUL} R1, b

MOV R2, c

MUL R2, d

DIV R1, R2

MOV x, R1

The main problem is that the compiler creates a new temporary at every reduction without considering value lifetime and register availability. The improved approach uses synthesized attributes, temporary lifetime analysis, and register reuse. For register $x = (a * b) / (c * d)$, the optimized code uses only two temporary locations / registers and avoids unnecessary register movements, making the generated code faster and more efficient.